rogii-wellbore-geology-prediction

3rd Place Solution

Rank	3
Team	tereka & Takoi
Author	tereka
Topic ID	733319
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733319
Retrieved with Kaggle CLI on 2026-08-07.

First of all, we would like to thank Kaggle and the competition hosts for organizing this excellent competition and providing such an interesting challenge. We are also deeply grateful to our team member @takoihiraokazu for their ideas, experiments, discussions, and persistence throughout the competition.

What made this competition especially enjoyable was the opportunity to explore an unusually diverse range of approaches—from probabilistic physics models such as HMMs and Particle Filters to GR-correlation models, neural sequence models, and dynamic ensembling. Investigating how these very different methods complemented one another was both technically challenging and genuinely fascinating.

Solution

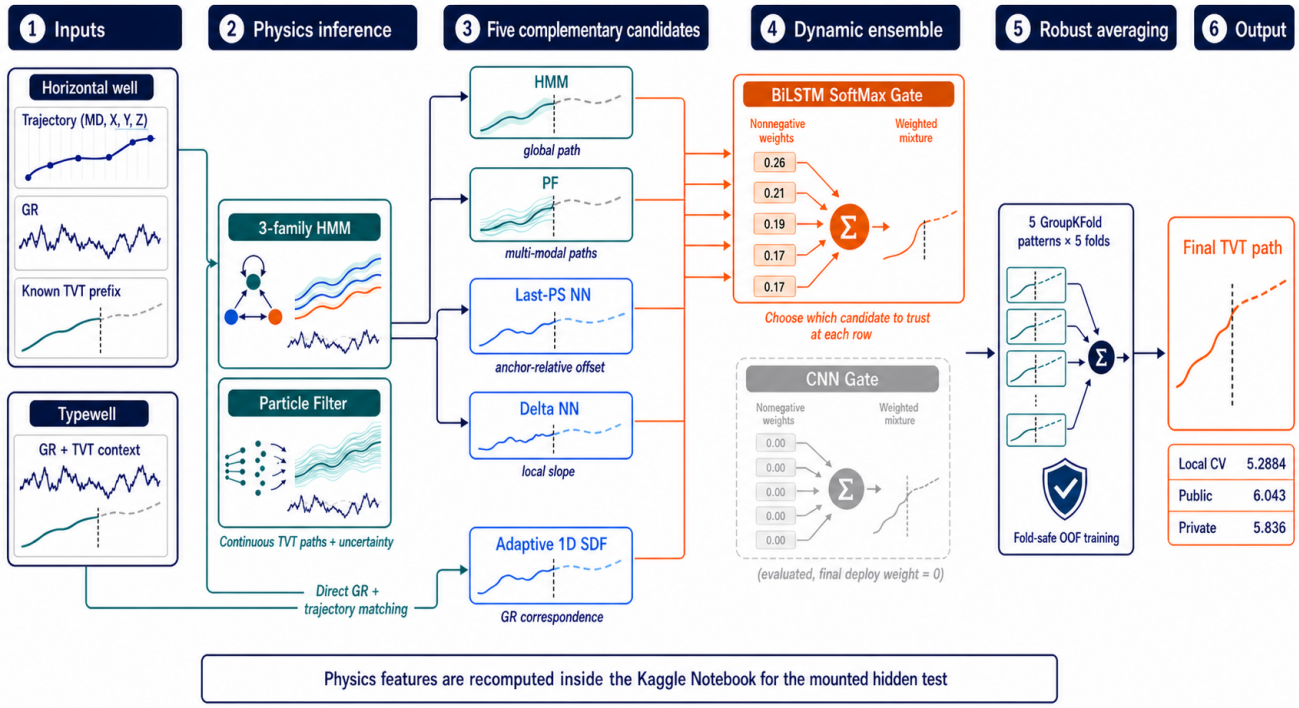
TL;DR

The core idea of our solution is to **generate several physically plausible TVT trajectories, complement them with neural models that make different kinds of errors, and combine the candidates at every position using SoftMax Gating**.

- **Physics models:** generate TVT trajectories from GR and well geometry using an HMM and a Particle Filter (PF).
- **Neural Network from Last PS:** predict the future absolute offset and uncertainty relative to the last known TVT immediately before Prediction Start (PS).
- **Delta Neural Network:** predict row-to-row TVT changes and integrate them to reconstruct a trajectory.
- **Adaptive 1D SDF:** directly match horizontal-well GR and trajectory against typewell GR/TVT candidates. It does not use HMM or PF predictions.
- **Ensemble:** dynamically mix five candidates—HMM, PF, two neural networks, and Adaptive 1D SDF—with a sequential SoftMax Gate.
- **Validation:** run five-fold GroupKFold by well under five different split patterns, giving 25 checkpoints per model family.

Our fold-safe Local CV used for model selection was **RMSE 5.2884**. The submitted notebook scored **6.043 Public / 5.836 Private**.

Solution Overview



Before PS, `TVT_input` is visible for each horizontal well; after PS, TVT is hidden. We can still use the well trajectory (`MD, X, Y, Z`), horizontal GR, and the associated typewell information (`TVT, GR, Geology`).

Pure time-series extrapolation accumulates even a small formation-dip error over a long horizontal section. Pure GR matching is also ambiguous because similar GR patterns, GR bias, missing values, and thickness changes can produce several plausible alignments. We therefore used three stages:

1. Match GR with HMM and PF while preserving trajectory continuity.
2. Use physics predictions, trajectory, GR, and typewell information in the neural networks to correct systematic physics-model errors.
3. Dynamically combine five complementary candidates with a SoftMax Gate.

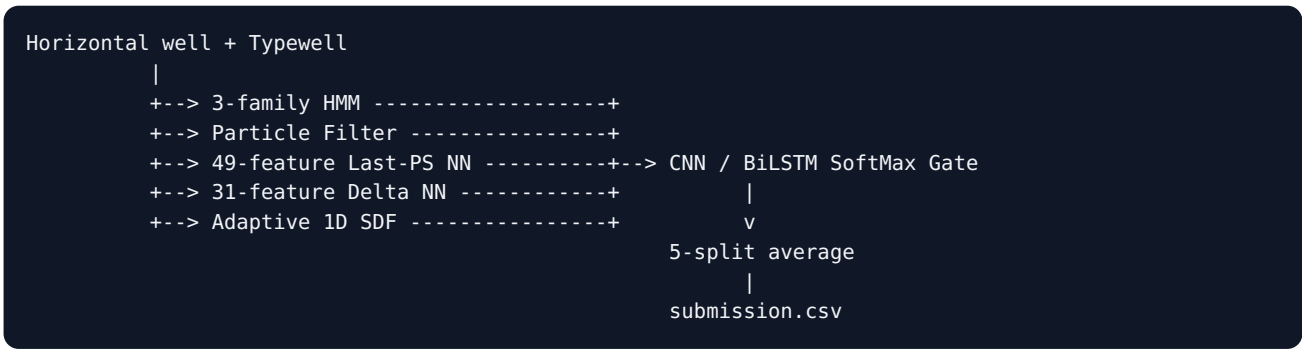
The table below summarizes the component-level Local CV and training losses. All CV values are RMSE, where lower is better. Except for the HMM mixture, standalone candidates and gates use OOF predictions from the default GroupKFold; the final ensemble reports the fold-safe CV used for model selection.

Because this was a notebook-only competition, standalone components were also rerun on the hidden test set inside Kaggle notebooks and evaluated as Code Submissions. Unmeasured results are shown as `–` rather than estimated.

Method	Training loss	Local CV	Public LB	Private LB
3-family HMM	None (probabilistic state estimation)	5.9703	6.207	6.229
Particle Filter	None (Sequential Monte Carlo)	6.5827	7.011	6.725
Neural Network from Last PS	Gaussian NLL	5.4723	6.074	5.903

Method	Training loss	Local CV	Public LB	Private LB
Delta Neural Network	Delta RMSE	5.7703	6.093	6.102
Adaptive 1D SDF	Candidate CE + 0.25 Smooth L1 + 0.10 Delta + 0.05 RNC	9.5415	8.502	8.837
CNN SoftMax Gate	Prediction MSE + best-candidate CE	5.4255	6.027	5.814
BiLSTM SoftMax Gate	Prediction MSE + best-candidate CE	5.3329	6.043	5.836
Final 5-split ensemble	Same as the BiLSTM Gate	**5.2884**	**6.043**	**5.836**

The final pipeline was:



Physics features were recomputed inside the Kaggle notebook for the mounted hidden test data. This made the pipeline reproducible even when the competition dataset mount or hidden-test contents changed, instead of depending on a fixed precomputed test-feature table.

Validation Strategy

GroupKFold by Well

Rows from the same well are strongly correlated, so a row-wise split causes severe leakage. We used five-fold GroupKFold with `well_id` as the group and always placed every row of one well in the same fold.

The training set contains 773 wells and 3,783,989 rows. A split contains `155 / 155 / 155 / 154 / 154` validation wells. We evaluated OOF predictions using row-level RMSE, matching the competition metric.

What “Fold-Safe” Means

In this solution, **fold-safe** means more than splitting by well. The entire pipeline is closed within each fold so that validation labels—or intermediate predictions from models trained on a validation well—cannot flow back into feature generation, normalization, or gate training.

We applied the following safeguards:

1. **Well-level split:** all rows from the same well remain in one fold, preventing row-wise leakage.
2. **Fold-local preprocessing:** feature means, standard deviations, and related statistics are fitted only on the training folds and then applied to the validation fold.

3. **OOF candidates:** training inputs for the NNs, Adaptive 1D SDF, and CNN/BiLSTM gates use only predictions from fold models that did not train on the corresponding validation well.
4. **Synchronized base-model and gate folds:** base models and gates use the same well assignment within each split, preventing the gate from seeing in-fold base predictions.
5. **OOF-only blending:** final CNN/BiLSTM weights and model-selection decisions are determined only from OOF predictions, never by fitting to the Public LB.
6. **Fold averaging at test time:** every fold model is applied to the hidden test set, followed by averaging across five folds and five split patterns.

For example, the NN candidate supplied for a gate-validation well `W` is generated by an NN trained without `W`. The gate itself is also evaluated without training on `W`. Without this two-stage OOF protocol, the gate could exploit in-fold base-model errors and produce an unrealistically optimistic Local CV.

Why 5 Splits × 5 Folds?

The two “fives” serve different purposes:

- **5 folds:** create leakage-free OOF predictions for every well and average five models at test time.
- **5 splits:** change the GroupKFold well assignment itself and reduce dependence on a particular fold composition.

We used `default, split101, split102, split202, split303`. Well labels were deterministically reordered from each seed before applying GroupKFold. Pairwise Adjusted Rand Index values between the five assignments ranged from approximately `-0.0013` to `0.0038`, indicating nearly independent assignments.

Each split trained NN49, NN31, Adaptive 1D SDF, CNN Gate, and BiLSTM Gate in five folds:

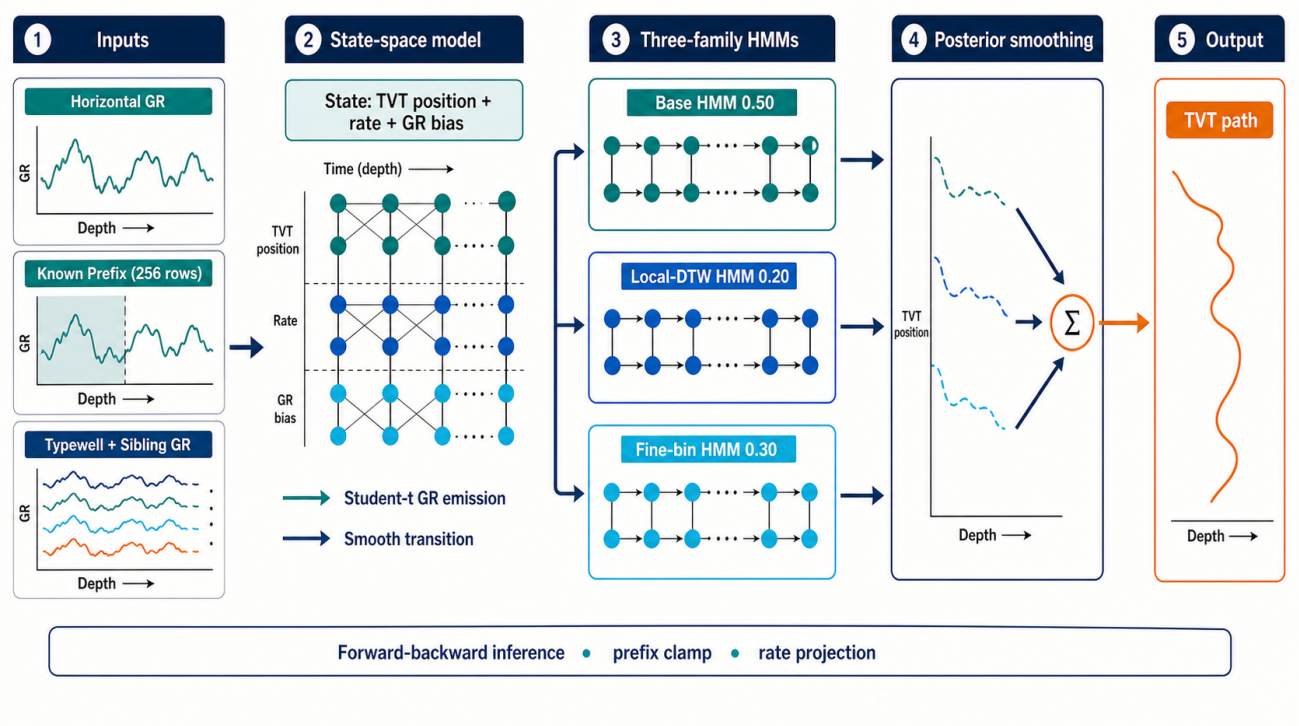
Model family	Splits	Folds	Checkpoints
Last-PS NN	5	5	25
Delta NN	5	5	25
Adaptive 1D SDF	5	5	25
CNN SoftMax Gate	5	5	25
BiLSTM SoftMax Gate	5	5	25

Within a split, the base models and gates shared the same fold assignment. This ensured that a gate-training row never contained a base prediction from a model trained on that well.

Increasing the ensemble from three to five split patterns improved the Private LB of the same Gaussian-NLL configuration from **5.903 to 5.836 (-0.067)**. Fold-safe Local CV also improved from **5.3069 to 5.2884 (-0.0184)**, so we selected the five-split version.

Physics Models

HMM



Instead of predicting each row independently, the HMM estimates the entire well as one continuous path. Its hidden state combines:

- **TVT position:** which typewell TVT corresponds to the current horizontal-well row.
- **Formation rate:** `d(TVT + Z) / dMD`.
- **GR bias:** a slowly changing offset between horizontal and reference GR.
- **Reference family:** a typewell-oriented or sibling-lateral-oriented reference curve.

Reference GR

Training wells identified as sharing the same typewell were grouped as siblings, and their lateral GR curves were aggregated into 0.25 ft TVT bins. The base HMM used two reference mixtures, with typewell/sibling weights of `0.2 / 0.8` in the two orientations, while reference identity remained part of the hidden state.

We also built a `(TVT_input, GR)` template from the target well's known prefix and mixed it into the HMM reference according to the number of observations. This self-reference exploits the fact that GR calibration is shared between the known prefix and future section.

Initial State and Prefix-in-Model

- Estimate the initial formation rate robustly from the last 256 known-prefix rows.
- Prepend the last 128 known-prefix rows to the HMM sequence.
- Clamp the prefix TVT states around the observed `TVT_input` values.

Compared with initialization from only the PS point, this carries the recent dip and GR alignment into the HMM posterior.

Emission and Transition

The GR emission used a Student-t distribution with one degree of freedom, making it more robust to outliers than a Gaussian likelihood. Transitions encouraged smooth changes in position, rate, and bias. Forward-backward smoothing then produced a posterior mean conditioned on the complete sequence.

Finally, we projected the predicted path using the 1st–99th percentile formation-rate range estimated from the last 256 known rows. The absolute rate was capped at 0.25, and the integrated TVT correction—not the correction per row—was limited to 10 ft.

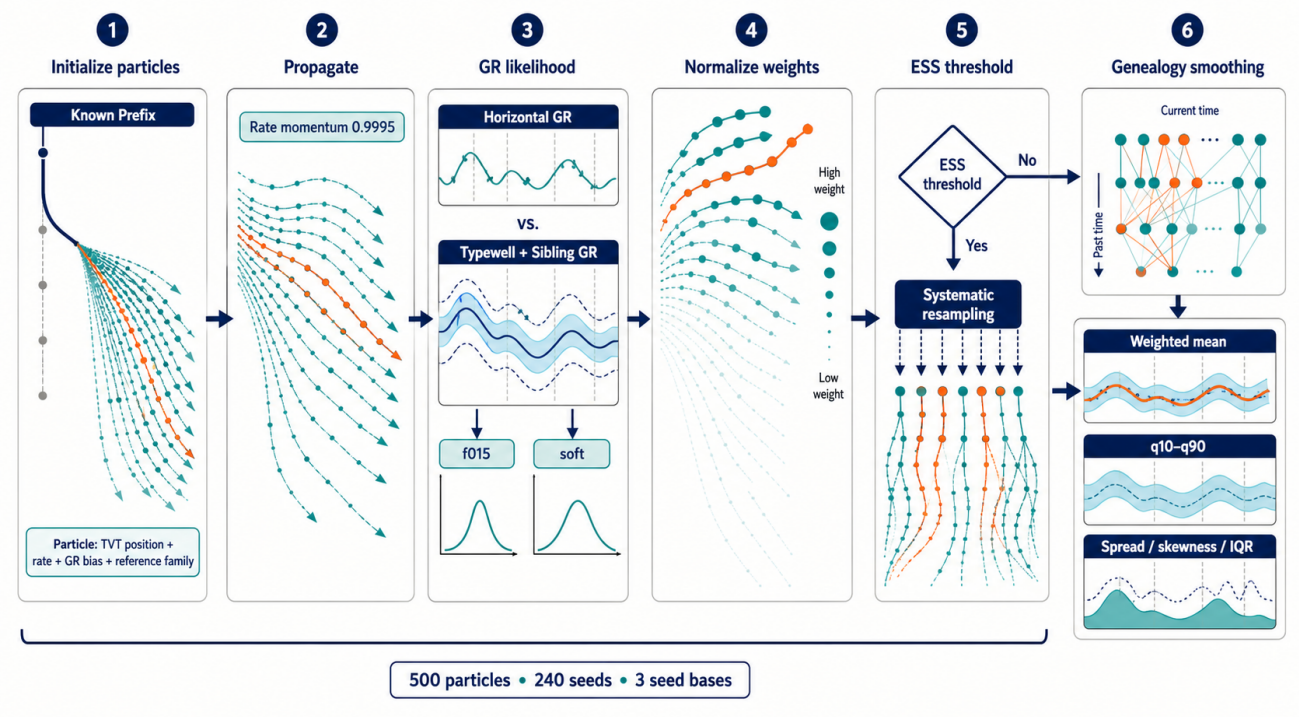
Three-Family HMM Ensemble

The final notebook mixed three HMM families with different error patterns:

HMM family	Weight	Main characteristic
Exp417 base HMM	0.50	Self-prefix, two references, bias state, exact forward-backward
Local-DTW HMM	0.20	Local GR-pattern matching with small stretch
Fine-bin HMM	0.30	Higher-resolution sibling reference using 0.0625 ft bins

Although Local-DTW was not the strongest standalone model, its lower error correlation with the base HMM made it valuable in the ensemble. The HMM-mixture OOF RMSE improved from **6.0492 to 5.9703**, while the combined HMMPF improved from **5.6648 to 5.6155**.

Particle Filter



The PF tracks many stochastic TVT trajectories, complementing the smoother HMM with multimodal hypotheses and uncertainty features.

Each particle state approximately contains:

- formation position `TVT + Z`;
- formation rate;
- GR bias;
- hard/soft observation family;
- typewell-oriented/sibling-oriented reference flag.

The main settings were 500 particles, 240 seeds, three seed bases, rate momentum 0.9995, rate noise 0.001, and position noise 0.005. At every row, we evaluated horizontal GR against reference-GR likelihoods and applied systematic resampling when the Effective Sample Size fell below its threshold. Reference and bias families were inherited after resampling.

We used two observation models:

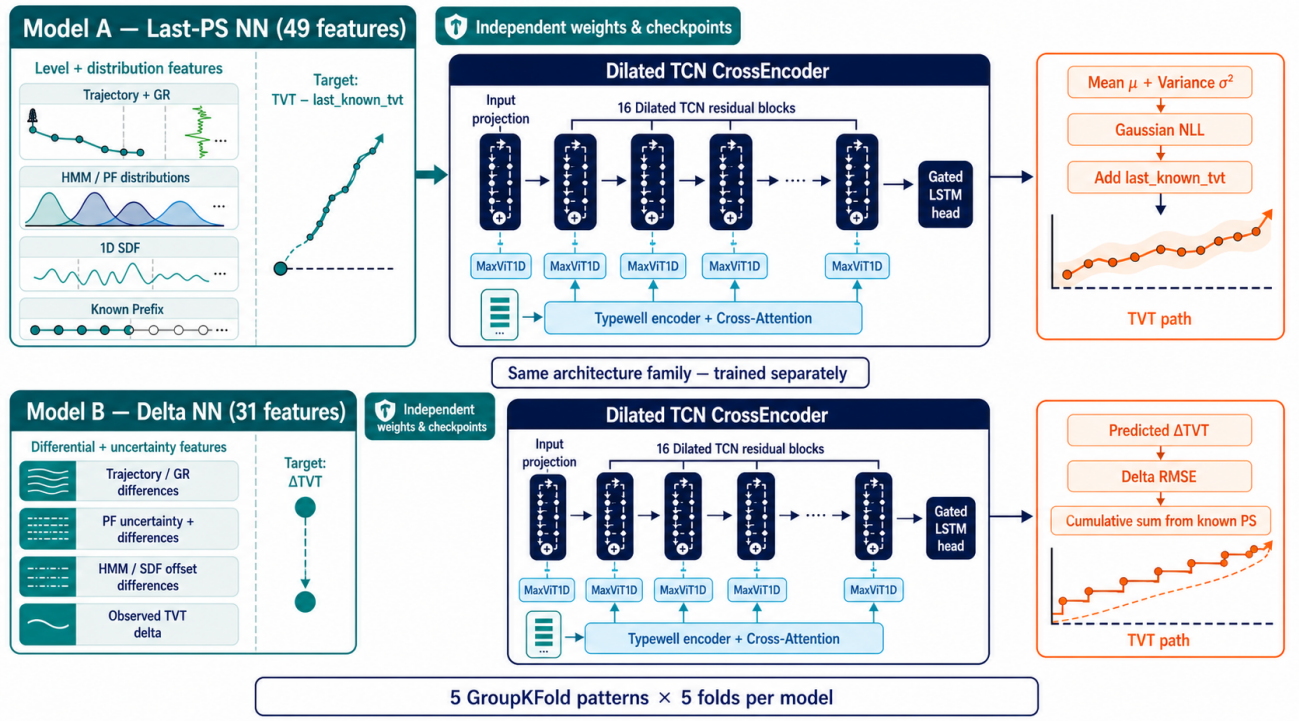
- **`f015:`** 15% of particles belong to a hard family whose likelihood is gradually strengthened near the beginning.
- **`soft:`** all particles allow GR bias.

Seed-level paths were weighted using full-sequence likelihood and smoothed by tracing particle genealogy backward. We stored the following as NN features:

- weighted mean;
- spread, skewness, and IQR;
- q10, q25, q50, q75, and q90;
- previous-row and next-row differences of those statistics.

The standalone PF candidate was `0.6 × f015 + 0.4 × soft`. The HMMPF physics feature used `0.6 × three-family HMM + 0.4 × PF`. The HMM contributes a smooth, stable global path, while the PF contributes alternative hypotheses and uncertainty.

Neural Networks



Neural Network from Last PS

This model anchors every hidden row at `last_known_tvt`, immediately before PS, and predicts:

`target = TVT - last_known_tvt`

Predicting an anchor-relative offset instead of absolute TVT removes much of the well-to-well TVT-level variation.

Features

The 49 features consist of:

- **Raw/mask:** `X, Y, Z, GR_filled, GR_missing, known_tvt_mask`.
- **PS-relative geometry:** `md_from_ps, z_from_ps, Z_diff_1`.
- **PF distribution:** f015/soft mean offsets, spread, skewness, IQR, and q10–q90.
- **Local dynamics:** previous/next differences for X/Y/Z/GR/TVT_input and PF statistics.
- **Physics candidates:** HMMPF offset, MultiGrid HMM offset, and their gap.
- **Correlation candidate:** anchor-relative Adaptive 1D SDF prediction.

Architecture

- Input projection: 96 channels.
- 16 full-resolution dilated-TCN residual blocks.
- Kernel size 7; dilation cycle `1,2,4,...,128`, repeated twice.
- MaxViT1D local-window/sparse-grid attention after blocks 4, 8, 12, and 16.
- Downsampled typewell cross-attention after block 8.

- Typewell encoder: residual encoder plus gated BiLSTM.
- Output head: gated one-layer LSTM.
- Gaussian head predicting both mean and variance.

The loss was Gaussian Negative Log-Likelihood over valid hidden rows:

$$L_{GNLL} = 0.5 \times [\log(\sigma^2) + (y - \mu)^2 / \sigma^2]$$

The model outputs μ and a raw variance, with $\sigma^2 = \text{softplus}(\text{raw_variance}) + 0.001$ to guarantee a variance floor. At inference, μ is used as the TVT candidate. The predictive standard deviation is not added to the candidate; it is supplied as a confidence feature to the SoftMax Gate.

We trained for 100 epochs with batch size 16, Scheduler-Free AdamW, learning rate 0.002, weight decay 1e-4, dropout 0.1, and EMA decay 0.995. We also masked short contiguous GR blocks during training.

Standalone OOF RMSE on the default split was **5.4723**. Across the three-split ensemble, switching the same NN from RMSE to Gaussian NLL improved RMSE from **5.4176 to 5.3645**.

Delta Neural Network

Whereas the Last-PS NN directly predicts a long-range offset, the Delta NN predicts:

$$\text{target}_t = \text{TVT}_t - \text{TVT}_{(t-1)}$$

The deltas are integrated from the known TVT at PS to reconstruct the path. This representation captures local slope changes well, but small biases accumulate over long distances, giving it a different error pattern from the Last-PS NN.

The 31 features emphasize differences and uncertainty rather than absolute levels:

- previous/next differences of X/Y/Z/GR/TVT_input;
- PF spread, skewness, IQR, and their adjacent differences;
- previous/next differences of HMMPF, MultiGrid, and Adaptive 1D SDF offsets;
- `TVT_input_observed_delta_last`.

The architecture belongs to the same dilated-TCN cross-encoder family as the Last-PS NN, but it is an independently trained model with a delta target. Its loss is delta RMSE over valid rows:

$$L_{delta} = \text{sqrt}(\text{mean}((\Delta\text{TVT} - \Delta\text{TVT_pred})^2))$$

We used dropout 0.5, weight decay 0.01, and manifold mixup on the head representation with probability 0.5 and `alpha=0.2`.

Default-split OOF RMSE was **5.7703**. Although weaker as a standalone model, it was retained because its local-slope representation was complementary. A Gaussian-NLL Delta pilot degraded performance, so the final model kept RMSE loss.

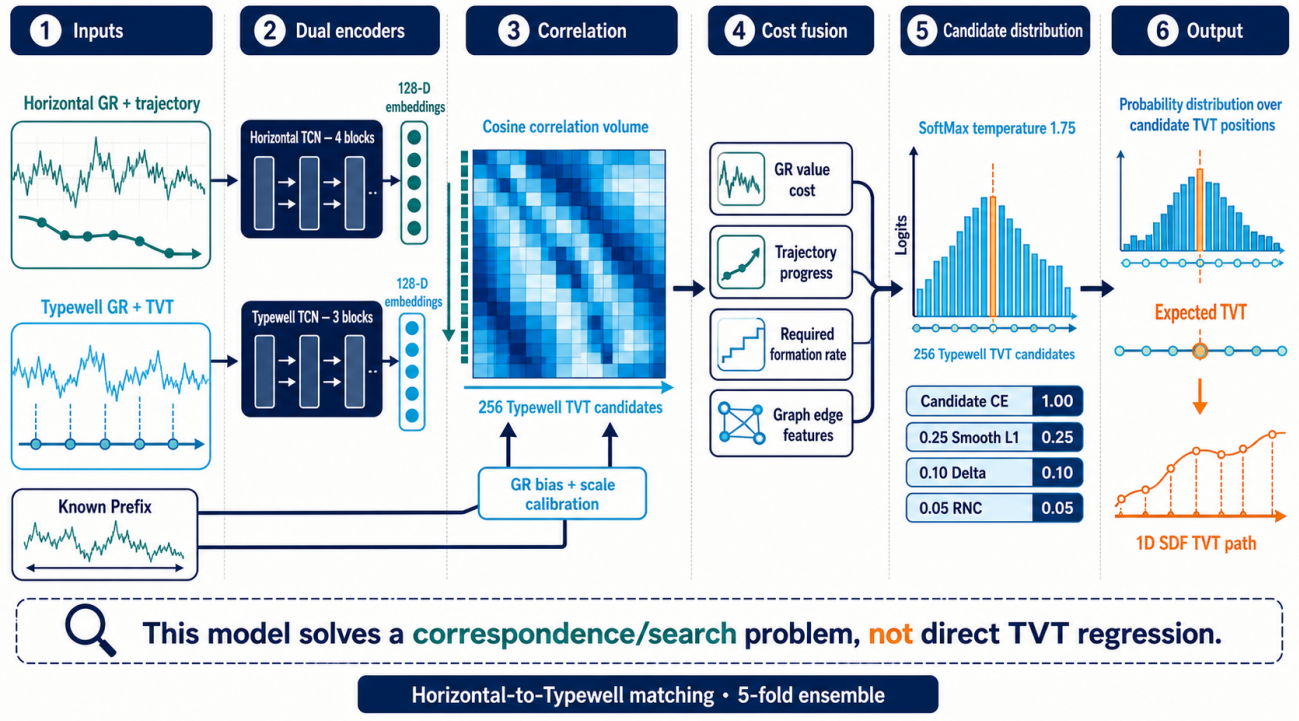
Training Augmentation

The two neural networks share an architecture family but use different augmentation suited to their targets:

- **Last-PS NN:** mask short contiguous GR blocks and pass the missing/mask indicators to the model. This discourages dependence on isolated GR peaks and encourages recovery from trajectory, physics candidates, and surrounding context.
- **Delta NN:** apply manifold mixup to the head representation with probability 0.5 and `alpha=0.2` for the Beta distribution. This smooths the local-delta representation and reduces overfitting to fine-scale slopes of individual wells.

Augmentation was applied only to training folds, never to validation OOF or test inference.

Adaptive 1D SDF



Adaptive 1D SDF is a correlation model that directly estimates which typewell TVT candidate corresponds to each horizontal-well row. **It is independent of the HMM and PF predictions**; the models meet only later in the SoftMax Gate.

- Encode horizontal GR and trajectory using a four-block TCN.
- Encode typewell GR/TVT context using a three-block TCN.
- Construct a query-key cosine-correlation volume with embedding dimension 128.
- Compare 256 TVT candidates on the typewell.
- Fuse GR-value cost with trajectory progress, required formation rate, and graph-edge features.
- Estimate well-specific GR-bias correlation and scale from the known prefix.
- Apply SoftMax with temperature 1.75 and output the expected TVT.

Its loss was:

`L_SDF = L_candidate_CE + 0.25 L_SmoothL1(TVT/10) + 0.10 L_delta + 0.05 L_RNC`

- **`L_candidate_CE`:** cross-entropy for the typewell candidate row nearest to the ground-truth TVT.
- **`L_SmoothL1`:** regression loss on expected TVT from the candidate SoftMax.
- **`L_delta`:** auxiliary loss for TVT differences along the packed sequence.
- **`L_RNC`:** Rank-N-Contrast loss that teaches the embedding the ordering induced by TVT distance.

We trained for 100 epochs with batch size 8, hidden size 64, dropout 0.5, and EMA 0.999.

Its standalone RMSE was weaker than the other candidates, but it solved the task through direct horizontal/typewell correspondence rather than physics-state estimation or direct regression. This increased candidate diversity, and the SoftMax Gate could select it locally even when its average weight was small.

Ensemble SoftMax Gating

The gate received five candidates:

1. Three-family HMM.
2. PF.
3. 49-feature Last-PS NN.
4. Adaptive 1D SDF.
5. 31-feature Delta NN.

For every row, the gate used not only candidate values but also:

- each candidate's offset from the last known TVT;
- deviation from the candidate mean;
- one-step candidate changes;
- candidate standard deviation and range;
- MD/Z distance from PS, MD/Z steps, and 3D step length;
- GR, GR difference, and GR-missing flag;
- normalized position within the hidden section;
- Gaussian predictive standard deviation from the Last-PS NN.

The gate outputs logits and applies SoftMax across candidates:

`prediction_t = Σ softmax(logits_t)_k × candidate_(t,k)`

The CNN Gate used hidden size 32, kernel size 7, and two residual blocks. The BiLSTM Gate used hidden size 32 and a one-layer bidirectional LSTM. Both were trained for eight epochs with:

`L_gate = MSE(y, Σ w_k × candidate_k) + CE(w, best_candidate)`

The first term is the regression error after convex mixing. The second is classification loss using the candidate with the smallest absolute error at that row as the target.

Why SoftMax Gating?

Fixed weights treat the area immediately after PS, long extrapolation ranges, missing-GR regions, and regions with large candidate disagreement identically. A SoftMax Gate can vary weights continuously using within-well context and candidate disagreement.

Nonnegative weights summing to one also constrain the output to a convex combination. With only 773 wells, this reduced extreme extrapolation compared with a high-capacity unconstrained stacker. We first generated physically plausible candidates and trained the gate to decide **which candidate to trust**, rather than generate an entirely new TVT trajectory.

The limitation is that a convex combination cannot correct drift shared by all candidates beyond their range. OOF oracle analysis confirmed this ceiling, so SoftMax represented a deliberate preference for robustness over maximum flexibility.

We also tested an unconstrained BiLSTM refiner that directly predicted `last_known_tvt - target_tvt` instead of mixing candidates. Gaussian NLL improved this direct regressor by 0.0140 over MSE, but its CV was **5.4023**, 0.1226 worse than the SoftMax BiLSTM's **5.2797**, and it lost in all five folds. Constraining the model to select among plausible candidates generalized better than allowing unrestricted correction outside their range.

We averaged the five-split CNN and BiLSTM predictions separately and selected the final blend using OOF only. The saved deployment weights were CNN 0 and BiLSTM 1, making the final submission effectively a **5 splits × 5 folds BiLSTM SoftMax Gate ensemble**.

Private-LB Insights

This section summarizes interventions that produced measured Private-LB improvements across our team's submission history, **whether or not they were included in the final notebook**. Comparisons from a different solution branch are explicitly identified. Ideas that improved only Public LB or Local CV are excluded.

1. Reference-GR Design Was the Most Important HMM Decision

The largest Private gains came not from small NN-architecture changes but from **which GR curves were used as HMM references**:

- use sibling-lateral GR from wells sharing the same typewell, not only the typewell itself;
- aggregate sibling GR in fine TVT bins;
- mix the target well's known-prefix GR as a self-reference;
- preserve reference families as separate HMM states or separate HMMs instead of averaging them away;
- add HMMs with different error structures, such as Local-DTW.

In another solution branch, switching to a spatial-distance-weighted sibling HMM improved Private LB from **5.817 to 5.691 (-0.126)**. Rational distance weighting improved it from **5.817 to 5.720 (-0.097)**. Although these were not identical to the linked notebook, they consistently showed that reference quality and diversity dominated Private generalization.

2. Using the Known Prefix Well

In one branch, expanding initial-rate estimation from the last 30 known rows to the last 256 improved Private LB from **5.854 to 5.812 (-0.042)**. The longer window was less sensitive to short-term fluctuation immediately before PS and gave a more stable estimate of the well-specific formation trend.

Prepending the known prefix inside the HMM and clamping its TVT state was also more stable than using only a last-value anchor. Long hidden sections made drift the dominant Private-test error, so small improvements in the starting condition accumulated across many rows.

3. Averaging Five GroupKFold Patterns

Increasing the same Gaussian-NLL configuration from three to five split patterns improved Private LB from **5.903 to 5.836 (-0.067)**. Adding OOF diversity across different well assignments transferred better to Private than selecting models against Public LB.

4. Physics Refinements That Helped Private Outside the Final Solution

In separate branches not necessarily included in the final notebook, adding Rao-Blackwellized PF bias improved Private LB from **5.691 to 5.688 (-0.003)**, while fold-specific HMM reference pools improved it from **5.726 to 5.718 (-0.008)**. The gains were small, but both support the same direction: adapt the reference or latent bias at the well level.

Our main takeaway is to prioritize **the right reference GR, stable initial-rate estimation, diverse HMM error modes, and well-level CV diversity** before adding model complexity. The neural networks and SoftMax Gate were powerful, but the physics pipeline provided the foundation for Private generalization.